

CẨM NANG KỸ THUẬT & HƯỚNG DẪN THỰC HÀNH HỌC THUẬT

Thiết kế, Phân tích Toán học, Phân chia Công việc và Phát triển Hệ thống Trực quan hóa Đồ thị

TÓM TẮT TÀI LIỆU (ABSTRACT):

Tài liệu này được biên soạn bởi Chuyên gia Giáo dục Công nghệ và Kỹ sư Lập trình Python cấp cao làm cẩm nang học thuật toàn diện cho dự án xây dựng ứng dụng trực quan hóa giải thuật đồ thị phục vụ giảng dạy môn Cấu trúc rời rạc. Tài liệu cung cấp cơ sở toán học chặt chẽ bằng ký hiệu Unicode trực quan, phân tích chi tiết ý tưởng cốt lõi và độ phức tạp của 9 thuật toán đồ thị cơ bản & nâng cao. Đồng thời, tài liệu cung cấp mã khung (Skeleton Code) chuẩn hóa kèm theo giải thích từng dòng code, bảng phân chia công việc tối ưu cho nhóm 5 người, và hướng dẫn chi tiết quy trình chuyển đổi mã nguồn thuần sang hoạt họa (Generator yield) tích hợp với mô hình xử lý bất đồng bộ của tkinter (tkinter.after). Đây là cẩm nang hướng dẫn chuẩn mực để triển khai hệ thống phần mềm giáo dục đạt chất lượng học thuật cao nhất.

MỤC LỤC TÀI LIỆU:

- I. Kiến trúc Hệ thống Trực quan hóa Đồ thị (Trang 2)
- II. Phân tích Toán học & Hướng dẫn Cài đặt 9 Giải thuật Cốt lõi (Trang 2-10)
 1. Duyệt đồ thị theo chiều rộng (BFS) (Trang 2-3)
 2. Duyệt đồ thị theo chiều sâu (DFS) (Trang 3-4)
 3. Kiểm tra đồ thị hai phía (Bipartite Graph) (Trang 4-5)
 4. Tìm đường đi ngắn nhất (Dijkstra) (Trang 5-6)
 5. Cây khung nhỏ nhất - Thuật toán Prim (Trang 6-7)
 6. Cây khung nhỏ nhất - Thuật toán Kruskal (Trang 7-8)
 7. Luồng cực đại trên mạng - Edmonds-Karp (Trang 8-9)
 8. Chu trình Euler - Thuật toán Fleury (Trang 9)
 9. Chu trình Euler - Thuật toán Hierholzer (Trang 10)
- III. Bảng phân công công việc nhóm 5 người (Trang 11)
- IV. Quy trình phối hợp, Pipeline CI & Tiêu chuẩn hoàn thành (DoD) (Trang 11-12)

I. KIẾN TRÚC HỆ THỐNG TRỰC QUAN HÓA ĐỒ THỊ

Hệ thống được thiết kế theo nguyên lý phân tách mối quan tâm (Separation of Concerns) để đảm bảo tính độc lập tối đa giữa logic nghiệp vụ (thuật toán đồ thị) và tầng hiển thị tương tác (GUI):

1. Tầng Utility (Biểu diễn đồ thị): Cài đặt trong `utils/conversions.py` để chuyển đổi tự động qua lại giữa Ma trận kề, Danh sách kề, Danh sách cạnh. Hỗ trợ cho cả đồ thị vô hướng và có hướng nhằm tối ưu dữ liệu đầu vào cho các thuật toán khác nhau.
2. Tầng Core Algorithm (Độc lập): Đặt trong `algorithms/pure/`. Các thuật toán ở đây chỉ thực hiện tính toán thuần túy trên cấu trúc dữ liệu cơ bản, không có bất kỳ thư viện giao diện nào, rất phù hợp để kiểm thử đơn vị (Unit Test) và chấm điểm tự động.
3. Tầng Generator (Animation): Đặt trong `algorithms/`. Đây là các hàm đồng hành cùng thuật toán thuần, được sửa đổi cấu trúc sử dụng từ khóa `yield` ở các điểm chốt của giải thuật nhằm trả dữ liệu trạng thái hiện tại (Đỉnh đang xét, Cạnh đang duyệt, Khoảng cách) về cho giao diện cập nhật thời gian thực.
4. Tầng Giao diện (Tkinter GUI): Nằm ở `gui.py`, lắng nghe các tương tác chuột để vẽ đồ thị, sau đó liên kết bộ lặp thời gian bất đồng bộ của Tkinter để lấy dữ liệu từ Generator vẽ hoạt họa.

II. PHÂN TÍCH TOÁN HỌC & HƯỚNG DẪN CÀI ĐẶT 9 GIẢI THUẬT CỐT LÕI

1. Duyệt đồ thị theo chiều rộng (BFS)

Ý tưởng cốt lõi: BFS hoạt động theo nguyên lý loang đều. Từ đỉnh nguồn s , thuật toán duyệt qua các đỉnh kề trực tiếp của s , sau đó mới di chuyển sang các đỉnh kề có khoảng cách xa hơn. Thuật toán sử dụng cấu trúc dữ liệu Hàng đợi (Queue) để quản lý thứ tự các đỉnh chuẩn bị duyệt theo nguyên tắc FIFO (Vào trước, ra trước).

Cơ sở Toán học:

Cho đồ thị $G = (V, E)$ và đỉnh nguồn $s \in V$. Khởi tạo mảng đã thăm: $visited[v] = False$ với mọi $v \in V$, riêng $visited[s] = True$.

Hàng đợi Q ban đầu chứa s . Ở mỗi bước, lấy u ra khỏi đầu Q . Với mỗi $v \in Adj(u)$:

Nếu $visited[v] == False \Rightarrow visited[v] = True$, đẩy v vào cuối Q , ghi nhận cạnh (u, v) thuộc cây duyệt BFS.

Mã nguồn thuật toán thuần (Pure BFS):

```
from collections import deque

def bfs(graph, start):
    visited = []
    visited_set = {start}
    queue = deque([start])
    traversal_edges = []
    while queue:
        u = queue.popleft() # Lay dinh u khoi dau queue
        visited.append(u)
        # Sorting de dam bao tinh deterministic (thu tu duyet dong nhac)
        for v, w in sorted(graph.get(u, []), key=lambda x: x[0]):
            if v not in visited_set:
                visited_set.add(v)
                queue.append(v) # Day dinh ke v vao cuoi queue
                traversal_edges.append((u, v, w))
    return visited, traversal_edges
```

Giải thích chi tiết mã nguồn:

- Dòng 1: Sử dụng collections.deque để tối ưu hóa thời gian POP ở đầu hàng đợi trong $O(1)$.
- Dòng 5: Đăng ký cấu trúc Queue lưu trữ đỉnh nguồn ban đầu.
- Dòng 10: Duyệt các đỉnh lân cận và sử dụng hàm sorted theo tên nhãn để kết quả duyệt không bị thay đổi phụ thuộc vào thứ tự nhập cạnh ban đầu, đảm bảo tính nhất quán (deterministic) khi chấm điểm.
- Dòng 13: Đẩy đỉnh kề chưa thăm vào cuối queue, ghi nhận vết cạnh duyệt đồ thị phục vụ trực quan hóa.

Đánh giá Độ phức tạp:

- Thời gian (Time Complexity): Tốt nhất = Xấu nhất = $O(V + E)$ vì thuật toán phải kiểm tra mọi đỉnh và mọi cạnh kề.
- Không gian (Space Complexity): $O(V)$ để lưu trữ hàng đợi Q và tập hợp visited_set chứa tối đa V đỉnh.

2. Duyệt đồ thị theo chiều sâu (DFS)

Ý tưởng cốt lõi: DFS đi theo nguyên lý tìm kiếm ưu tiên chiều sâu. Xuất phát từ một đỉnh nguồn s, thuật toán chọn đi xa nhất có thể dọc theo mỗi nhánh của đồ thị trước khi thực hiện quay lui (Backtracking).

Thuật toán sử dụng cơ chế Ngăn xếp (Stack), thường được cài đặt một cách tự nhiên thông qua Đệ quy hệ thống.

Cơ sở Toán học:

Cho đồ thị $G = (V, E)$ và đỉnh nguồn s. Đặt $visited[v] = False$ với mọi $v \in V$.

Định nghĩa hàm đệ quy DFS(u): đánh dấu $visited[u] = True$. Với mỗi $v \in Adj(u)$ sao cho $visited[v] == False$, ghi nhận cạnh (u, v) thuộc cây duyệt DFS, sau đó gọi đệ quy DFS(v).

Mã nguồn thuật toán thuần (Pure DFS):

```
def dfs(graph, start):
    visited = []
    visited_set = set()
    traversal_edges = []

    def dfs_recursive(u):
        visited_set.add(u)
        visited.append(u)
        # Sắp xếp đỉnh kề để kết quả nhất quán
        for v, w in sorted(graph.get(u, []), key=lambda x: x[0]):
            if v not in visited_set:
                traversal_edges.append((u, v, w))
                dfs_recursive(v) # Gọi đệ quy đi sâu xuống đỉnh v

    dfs_recursive(start)
    return visited, traversal_edges
```

Giải thích chi tiết mã nguồn:

- Dòng 5: Định nghĩa hàm đệ quy lồng (closure) `dfs_recursive` để truy cập trực tiếp các biến chung mà không cần truyền tham chiếu đồ thị.
- Dòng 11: Ghi nhận vết cạnh đi xuôi theo chiều sâu trước khi thực hiện gọi đệ quy ở dòng tiếp theo.
- Dòng 12: Gọi đệ quy chuyển quyền điều khiển sang đỉnh kề `v`, cơ chế stack hệ thống sẽ lưu vết đỉnh `u` để quay lui khi nhánh `v` duyệt xong.

Đánh giá Độ phức tạp:

- Thời gian (Time Complexity): Tốt nhất = Xấu nhất = $O(V + E)$ do phải viếng thăm mọi đỉnh và duyệt qua danh sách kề.
- Không gian (Space Complexity): $O(V)$ phụ thuộc vào độ sâu tối đa của ngăn xếp đệ quy trong trường hợp đồ thị dạng đường thẳng.

3. Kiểm tra đồ thị hai phía (Bipartite Graph Check)

Ý tưởng cốt lõi: Sử dụng giải thuật tô 2 màu (2-Coloring). Ta xuất phát từ một đỉnh chưa tô màu, tô màu 0. Duyệt qua các đỉnh kề của nó và tô màu nghịch đảo là 1. Nếu phát hiện một đỉnh kề đã được tô màu và màu của nó trùng với màu của đỉnh hiện tại, đồ thị không phải là hai phía. Để tăng tính học thuật, thuật toán thực hiện truy vết mảng cha để tìm ra chu trình lẻ (odd cycle) làm minh chứng phản ví dụ.

Cơ sở Toán học:

Đồ thị $G = (V, E)$ là đồ thị hai phía khi và chỉ khi G không chứa chu trình độ dài lẻ.

Hàm tô màu $c: V \rightarrow \{0, 1\}$. Nếu tồn tại cạnh $(u, v) \in E$ sao cho $c(u) == c(v)$ thì đồ thị chứa chu trình lẻ độ dài $2k + 1$ kết nối u, v và đỉnh tổ tiên chung gần nhất.

Mã nguồn thuật toán thuần (Bipartite Check):

```
def check_bipartite(graph):
    color = {} # Map lưu màu của từng đỉnh
    parent = {}
    for start_node in graph:
        if start_node not in color:
            color[start_node] = 0
            stack = [(start_node, 0)]
            while stack:
                u, c = stack[-1]
                unvisited_found = False
                for v, _ in graph.get(u, []):
                    if v not in color:
                        color[v] = 1 - c # Tô màu ngược lại cho đỉnh kề
                        parent[v] = u
                        stack.append((v, 1 - c))
                        unvisited_found = True
                        break
                elif color[v] == color[u]:
                    # Phát hiện xung đột, truy vết lấy chu trình lẻ
                    cycle = [v, u]
                    curr = u
                    while curr in parent and curr != v:
                        curr = parent[curr]
                        cycle.append(curr)
                    return False, cycle[::-1]
            if not unvisited_found: stack.pop()
    return True, color
```

Giải thích chi tiết mã nguồn:

- Dòng 11: Sử dụng kỹ thuật tô màu đảo 1 - c (nếu màu hiện tại là 0 thì tô 1, ngược lại nếu là 1 thì tô 0).
- Dòng 16: Khi phát hiện `color[v] == color[u]`, ta tìm thấy cạnh vi phạm điều kiện đồ thị hai phía.
- Dòng 18-21: Dùng mảng `parent` đi ngược từ `u` về `v` để thu thập danh sách các đỉnh tạo nên chu trình lẻ gây mâu thuẫn làm minh chứng hiển thị lên giao diện.

Đánh giá Độ phức tạp:

- Thời gian (Time Complexity): $O(V + E)$ do duyệt qua các đỉnh và cạnh để tô màu.
- Không gian (Space Complexity): $O(V)$ để lưu trữ mảng màu và cây khung cha của mảng `parent`.

4. Tìm đường đi ngắn nhất (Dijkstra)

Ý tưởng cốt lõi: Dijkstra giải bài toán đường đi ngắn nhất từ một nguồn trên đồ thị có trọng số không âm.

Thuật toán duy trì tập các đỉnh đã tìm ra đường đi ngắn nhất cố định. Ở mỗi bước, chọn một đỉnh `u` chưa cố định có khoảng cách ước lượng nhỏ nhất, cố định nó, và thực hiện nới lỏng các cạnh kề của `u`.

Cơ sở Toán học (Nguyên lý nới lỏng - Relaxation):

Gọi $d[v]$ là khoảng cách ngắn nhất ước lượng từ nguồn `S` tới `v`. Với mọi cạnh $(u, v) \in E$:

Nếu $d[u] + w(u, v) < d[v]$ ⇨ Cập nhật $d[v] = d[u] + w(u, v)$ và đặt $prev[v] = u$.

Để tối ưu hóa bước tìm $\min\{d[u]\}$, sử dụng cấu trúc Min-Heap với khóa là $d[u]$.

Mã nguồn thuật toán thuần (Dijkstra):

```
import heapq

def dijkstra(graph, start):
    dist = {node: float('inf') for node in graph}
    prev = {node: None for node in graph}
    dist[start] = 0
    pq = [(0, start)] # Min-heap chứa cặp (khoảng_cách, đỉnh)
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue # Đỉnh đã được tối ưu từ trước, bỏ qua
        for v, w in graph.get(u, []):
            if w < 0:
                raise ValueError("Đồ thị chưa cạnh âm, Dijkstra không hỗ trợ!")
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
                heapq.heappush(pq, (dist[v], v))
    return dist, prev
```

Giải thích chi tiết mã nguồn:

- Dòng 7: Hàm `heapq.heappop` lấy ra đỉnh u có đường đi ngắn nhất tạm thời trong $O(\log V)$ thời gian.
- Dòng 8: Kiểm tra $d > \text{dist}[u]$ để loại bỏ các trạng thái cũ đã được tối ưu trước đó trong hàng đợi.
- Dòng 11-12: Kiểm tra lỗi trọng số âm để dừng chương trình một cách an toàn, tránh vòng lặp vô hạn.
- Dòng 13-16: Thực hiện phép nới lỏng và đẩy cập nhật mới vào heap.

Đánh giá Độ phức tạp:

- Thời gian (Time Complexity): $O(E \log V)$ do mỗi cạnh được nới lỏng tối đa một lần và chi phí heap là $\log V$.
- Không gian (Space Complexity): $O(V)$ để lưu trữ khoảng cách, vết và cấu trúc dữ liệu của Min-Heap.

5. Cây khung nhỏ nhất - Thuật toán Prim

Ý tưởng cốt lõi: Thuật toán Prim tiếp cận bài toán tìm cây khung nhỏ nhất (MST) theo hướng duyệt đỉnh. Bắt đầu từ một đỉnh nguồn bất kỳ, cây khung được mở rộng bằng cách liên tục chọn cạnh có trọng số nhỏ nhất kết nối giữa một đỉnh đã nằm trong cây với một đỉnh chưa nằm trong cây.

Cơ sở Toán học:

Gọi T là tập các đỉnh đã kết nạp vào cây khung (ban đầu $T = \{\text{start}\}$).

Tại mỗi bước, chọn cạnh $(u, v) \in E$ sao cho $u \in T, v \notin T$ và $w(u, v) = \min\{w(x, y) \mid x \in T, y \notin T\}$.

Đưa v vào T , thêm cạnh (u, v) vào tập cạnh MST. Lặp lại cho đến khi T chứa tất cả các đỉnh.

Mã nguồn thuật toán thuần (Prim):

```
import heapq

def prim(node_count, graph, start=0):
    visited = {start}
    mst = []
    total_weight = 0
    pq = [] # Heap lưu các cạnh ứng viên dạng (trọng_số, u, v)
    for v, w in graph.get(start, []):
        heapq.heappush(pq, (w, start, v))
    while pq and len(visited) < node_count:
        w, u, v = heapq.heappop(pq)
        if v not in visited:
            visited.add(v)
            mst.append((u, v, w))
            total_weight += w
            for next_v, next_w in graph.get(v, []):
                if next_v not in visited:
                    heapq.heappush(pq, (next_w, v, next_v))
    return mst, total_weight
```

Giải thích chi tiết mã nguồn:

- Dòng 7-8: Đẩy tất cả các cạnh kề của đỉnh gốc xuất phát vào Min-Heap làm tập ứng viên ban đầu.
- Dòng 11: Lấy ra cạnh có trọng số nhỏ nhất. Kiểm tra `v` not in `visited` để tránh kết nạp cạnh tạo chu trình với các đỉnh đã nằm trong cây.
- Dòng 15-17: Khi đưa `v` vào cây, bổ sung các cạnh kết nối từ `v` tới các đỉnh bên ngoài cây vào Heap.

Đánh giá Độ phức tạp:

- Thời gian (Time Complexity): $O(E \log V)$ vì mỗi đỉnh được duyệt một lần và các cạnh được thêm/bớt khỏi Heap.
- Không gian (Space Complexity): $O(V + E)$ để lưu trữ thông tin các cạnh kề trong Heap.

6. Cây khung nhỏ nhất - Thuật toán Kruskal

Ý tưởng cốt lõi: Thuật toán Kruskal tiếp cận tìm MST theo hướng duyệt cạnh. Thuật toán sắp xếp toàn bộ cạnh của đồ thị theo trọng số tăng dần, sau đó duyệt qua từng cạnh để thêm vào cây khung nếu cạnh đó không tạo thành chu trình với các cạnh đã chọn. Để kiểm tra chu trình tối ưu, ta dùng cấu trúc dữ liệu Disjoint Set Union (DSU).

Cơ sở Toán học (DSU & Path Compression):

Mỗi đỉnh đại diện cho một tập hợp. Hàm `find(x)` trả về đại diện nhóm của `x` kết hợp nén đường đi:

`parent[x] = find(parent[x])` giúp độ sâu cây giảm còn xấp xỉ hằng số.

Cạnh (u, v) được kết nạp khi và chỉ khi `find(u) != find(v)`. Sau đó, thực hiện `union(u, v)` bằng cách đặt `parent[find(u)] = find(v)`.

Mã nguồn thuật toán thuần (Kruskal):

```
def kruskal(node_count, edges):
    parent = list(range(node_count))
    def find(x):
        if parent[x] != x:
            parent[x] = find(parent[x]) # Path compression (Nén đường đi)
        return parent[x]
    def union(x, y):
        root_x, root_y = find(x), find(y)
        if root_x != root_y:
            parent[root_x] = root_y # Union
            return True
        return False
    sorted_edges = sorted(edges, key=lambda e: e[2])
    mst = []
    total_weight = 0
    for u, v, w in sorted_edges:
        if union(u, v):
            mst.append((u, v, w))
            total_weight += w
            if len(mst) == node_count - 1:
                break
    return mst, total_weight
```

Giải thích chi tiết mã nguồn:

- Dòng 4-5: Hàm find thực hiện đệ quy tìm gốc của phần tử, đồng thời gán trực tiếp cha của nút con về nút gốc đại diện (Path Compression), giảm chi phí truy vấn lần sau về $O(\alpha(V)) \approx O(1)$.
- Dòng 13: Sắp xếp toàn bộ danh sách cạnh theo trọng số tăng dần làm cơ sở cho tiếp cận tham lam.
- Dòng 20: Dừng thuật toán sớm khi đã tìm đủ $V - 1$ cạnh, giúp tối ưu hiệu năng đối với đồ thị dày cạnh.

Đánh giá Độ phức tạp:

- Thời gian (Time Complexity): $O(E \log E)$ phụ thuộc vào thời gian sắp xếp danh sách các cạnh.
- Không gian (Space Complexity): $O(V)$ để lưu trữ mảng parent quản lý các tập hợp liên thông của DSU.

7. Luồng cực đại trên mạng - Edmonds-Karp

Ý tưởng cốt lõi: Thuật toán Edmonds-Karp tìm luồng cực đại bằng cách liên tục tìm đường tăng luồng ngắn nhất từ nguồn s tới đích t trên đồ thị dư bằng thuật toán duyệt BFS. Sau đó, nó nới lỏng luồng bằng giá trị bottleneck nhỏ nhất tìm được trên đường đi này và cập nhật lại dung lượng trên các cạnh xuôi và ngược của đồ thị dư.

Cơ sở Toán học (Mạng dư & Đường tăng luồng):

Mạng dư G_f chứa các cạnh có dung lượng dư $r(u, v) = c(u, v) - f(u, v) > 0$.

BFS tìm đường đi P từ s đến t có số cạnh ít nhất trên G_f . Bottleneck $\Delta f = \min\{r(u, v) \mid (u, v) \in P\}$.

Với mỗi cạnh $(u, v) \in P$: cập nhật $f(u, v) = f(u, v) + \Delta f$ và $f(v, u) = f(v, u) - \Delta f$.

Mã nguồn thuật toán thuần (Edmonds-Karp):

```
from collections import deque

def bfs_augmenting_path(graph, capacity, flow, s, t, parent):
    visited = {s}
    queue = deque([s])
    while queue:
        u = queue.popleft()
        for v in graph[u]:
            residual = capacity[(u, v)] - flow.get((u, v), 0)
            if v not in visited and residual > 0:
                visited.add(v)
                parent[v] = u
                if v == t: return True
                queue.append(v)
    return False

def edmonds_karp(node_count, edges, s, t, is_directed=False):
    graph = {i: set() for i in range(node_count)}
    capacity = {}
    for u, v, w in edges:
        graph[u].add(v)
        capacity[(u, v)] = capacity.get((u, v), 0) + w
        if not is_directed:
            graph[v].add(u)
            capacity[(v, u)] = capacity.get((v, u), 0) + w
    flow = {}
    max_flow_val = 0
    parent = {}
    while bfs_augmenting_path(graph, capacity, flow, s, t, parent):
        path_flow = float('inf')
        curr = t
        while curr != s:
            p = parent[curr]
            residual = capacity[(p, curr)] - flow.get((p, curr), 0)
            path_flow = min(path_flow, residual)
            curr = p
        curr = t
        while curr != s:
            p = parent[curr]
            flow[(p, curr)] = flow.get((p, curr), 0) + path_flow
            flow[(curr, p)] = flow.get((curr, p), 0) - path_flow
            curr = p
        max_flow_val += path_flow
        parent = {}
    return max_flow_val, flow
```

Giải thích chi tiết mã nguồn:

- Dòng 3-14: Hàm `bfs_augmenting_path` tìm đường tăng luồng ngắn nhất trên đồ thị dư. Chỉ duyệt qua các cạnh có `residual > 0`.
- Dòng 31-36: Duyệt ngược qua mảng `parent` từ `t` về `s` để tìm giá trị dung lượng dư nhỏ nhất (bottleneck) trên đường đi.
- Dòng 38-43: Thực hiện cập nhật luồng: cộng luồng thực tế cho chiều đi và trừ luồng (luồng ảo) cho chiều ngược lại để cho phép giải thuật tự sửa sai ở các bước lặp sau.

Đánh giá Độ phức tạp:

- Thời gian (Time Complexity): $O(V E^2)$ thời gian. Việc sử dụng BFS đảm bảo số đường tăng luồng tối đa là $O(VE)$, mỗi lần tìm mất $O(E)$.
- Không gian (Space Complexity): $O(V + E)$ để lưu trữ cấu trúc mạng và các bản đồ luồng/dung lượng cạnh.

8. Chu trình Euler - Thuật toán Fleury

Ý tưởng cốt lõi: Thuật toán Fleury xây dựng chu trình Euler bằng cách duyệt tham lam trên các cạnh. Từ đỉnh hiện tại, ta chọn một cạnh chưa đi qua để di chuyển sang đỉnh tiếp theo. Quy tắc cốt lõi là ta phải **tránh đi qua cạnh cầu** (bridge) của phần đồ thị chưa duyệt còn lại, trừ khi ta không còn sự lựa chọn nào khác.

Cơ sở Toán học (Khái niệm cạnh cầu):

Cạnh $e = (u, v)$ thuộc đồ thị liên thông G là cạnh cầu khi và chỉ khi xóa e làm tăng số thành phần liên thông của G .

Fleury kiểm tra tính cầu bằng cách chạy DFS đếm số đỉnh có thể đi tới trước và sau khi xóa cạnh tạm thời.

Nếu `count_before > count_after` $\square$ e là cầu. Ta chỉ đi qua e khi bậc của u bằng 1.

Mã nguồn thuật toán kiểm tra cầu (Fleury Helper):

```
# Phương thức hỗ trợ kiểm tra cạnh cầu của Thành viên 5
def is_bridge(u, v, graph):
    # Đếm số đỉnh liên thông từ u trên đồ thị hiện tại
    count1 = dfs_count_reachable(u, graph)

    # Xóa cạnh (u, v) và kiểm tra lại
    remove_edge(u, v, graph)
    count2 = dfs_count_reachable(u, graph)

    # Khôi phục cạnh
    add_edge(u, v, graph)

    # Nếu số đỉnh reachable giảm đi, (u, v) là cạnh cầu
    return count1 > count2
```

9. Chu trình Euler - Thuật toán Hierholzer

Ý tưởng cốt lõi: Hierholzer là một giải thuật cực kỳ hiệu quả để tìm chu trình/đường đi Euler bằng cách lồng ghép các chu trình con đơn giản. Xuất phát từ một đỉnh, ta đi tự do qua các cạnh chưa duyệt đến khi tạo thành một chu trình khép kín, đẩy các đỉnh đi qua vào Ngăn xếp (Stack). Nếu đỉnh ở đỉnh Stack vẫn còn cạnh chưa duyệt, ta tiếp tục đi để tìm chu trình con mới và lồng vào chu trình lớn bằng cách pop Stack.

Cơ sở Toán học:

Vì mỗi đỉnh trong chu trình Euler đều có bậc chẵn, nên khi ta đi tự do và xóa các cạnh đã đi qua, ta chắc chắn sẽ quay về đỉnh xuất phát ban đầu tạo thành chu trình khép kín. Các cạnh còn dư trên đồ thị sẽ tạo thành các chu trình con liên kết với chu trình chính. Thuật toán hoạt động trong thời gian tuyến tính $O(E)$.

Mã nguồn thuật toán thuần (Hierholzer):

```
def hierholzer(graph, start):
    # graph: Adjacency list biểu diễn đồ thị
    adj = {u: [v for v, w in graph[u]] for u in graph} # Copy danh sách cạnh
    stack = [start]
    circuit = []
    while stack:
        u = stack[-1]
        if adj[u]:
            v = adj[u].pop(0) # Lấy và xóa cạnh khỏi đồ thị
            if u in adj[v]:
                adj[v].remove(u) # Xóa chiều ngược lại (đối với đồ thị vô hướng)
            stack.append(v)
        else:
            # Nếu đỉnh u không còn cạnh kề chưa duyệt, pop ra ghi nhận vào kết quả
            circuit.append(stack.pop())
    return circuit[::-1]
```

Giải thích chi tiết mã nguồn:

- Dòng 3: Sao chép cấu trúc cạnh ra biến cục bộ adj để thực hiện xóa dần cạnh khi đi qua.
- Dòng 9-11: Xóa cạnh hai chiều đối với đồ thị vô hướng ngay lập tức để tránh duyệt lại cạnh đó.
- Dòng 14: Khi đỉnh u hết cạnh kề khả dụng, ta thực hiện trích xuất đỉnh u khỏi Stack đưa vào mảng kết quả. Mảng kết quả đảo ngược chính là chu trình Euler khép kín cần tìm.

Đánh giá Độ phức tạp:

- Thời gian (Time Complexity): $O(E)$ do duyệt qua mỗi cạnh đúng một lần để thêm/xóa.
- Không gian (Space Complexity): $O(V + E)$ để lưu trữ bản sao danh sách kề và ngăn xếp Stack.

III. BẢNG PHÂN CÔNG CÔNG VIỆC NHÓM 5 NGƯỜI

Bảng dưới đây thiết lập chi tiết phân công công việc cân bằng cho cả 5 thành viên. Mỗi người phụ trách một phân hệ chức năng bao gồm thiết kế thuật toán thuần, thuật toán hoạt họa generator và lập trình tích hợp hiển thị GUI.

Thành viên	Mô-đun phụ trách	Tập tin mã nguồn	Đầu ra giao diện (GUI Output)
Thành viên 1	Quản lý biểu diễn đồ thị & Duyệt cơ bản	- utils/conversions.py - algorithms/traversal.py - algorithms/pure/traversal.py	Bảng biểu diễn ma trận/danh sách kề; hoạt họa tô màu đỉnh/cạnh duyệt BFS/DFS.
Thành viên 2	Đường đi ngắn nhất & Đồ thị hai phía	- algorithms/dijkstra.py - algorithms/bipartite.py - algorithms/pure/...	Vẽ đường đi ngắn nhất màu đỏ; tô màu 2 phân hoạch (Hồng/Xanh) hoặc highlight chu trình lẻ.
Thành viên 3	Cây khung nhỏ nhất (MST)	- algorithms/prim.py - algorithms/kruskal.py - algorithms/pure/...	Highlight cạnh MST đỏ đậm; hiển thị bảng sắp xếp cạnh (Kruskal) và thông số tổng trọng số.
Thành viên 4	Luồng cực đại trên mạng	- algorithms/max_flow.py - algorithms/pure/max_flow.py - Hỗ trợ gui.py	Vẽ nhãn dòng luồng flow/capacity trên cạnh; highlight đường tăng luồng; khung Canvas chính.
Thành viên 5	Đường đi & Chu trình Euler	- algorithms/fleury.py - algorithms/hierholzer.py - algorithms/pure/...	Chạy nét chu trình Euler động; hiển thị ngăn xếp Stack (Hierholzer) ở góc log.

IV. QUY TRÌNH PHỐI HỢP, PIPELINE CI & TIÊU CHUẨN HOÀN THÀNH

1. Quy trình chuyển đổi từ mã nguồn thuần sang Hoạt họa Generator

Để giao diện không bị treo khi thực thi thuật toán lâu, hệ thống không sử dụng vòng lặp vô hạn hay hàm trì hoãn đồng bộ (time.sleep). Thay vào đó, mỗi thuật toán thuần được chuyển đổi sang dạng Generator bằng cách thay các điểm chốt trạng thái bằng từ khóa yield.

Minh họa mẫu chuyển đổi sang Generator:

```
# Ví dụ cách chuyển đổi sang Generator trong algorithms/dijkstra.py
def dijkstra_generator(graph, start):
    dist = {node: float('inf') for node in graph}
    dist[start] = 0
    pq = [(0, start)]
    yield "START", {"dist": dist.copy(), "curr": start}

    while pq:
        d, u = heapq.heappop(pq)
        yield "EXAMINE_NODE", {"node": u, "dist": dist.copy()}
        ...
        # Sau mỗi lần nói long cạnh v thành công:
        yield "RELAX_EDGE", {"edge": (u, v), "dist": dist.copy()}
    yield "FINISHED", {"dist": dist, "prev": prev}
```

2. Cơ chế tích hợp hoạt họa với vòng lặp bất đồng bộ của tkinter (after)

Giao diện Tkinter chạy đơn luồng. Để thực hiện vẽ hoạt họa từng bước, GUI sẽ khởi tạo Generator của thuật toán và đăng ký một hàm lặp lại sau mỗi khoảng thời gian delay thông qua hàm tkinter.after:

```
# Tích hợp trong gui.py
def run_animation_step(self):
    try:
        # Lấy trạng thái tiếp theo từ generator
        step_type, data = next(self.current_generator)
        self.redraw_graph_canvas(step_type, data)
        self.update_log_panel(step_type, data)

        if step_type != "FINISHED":
            # Đăng ký gọi lại chính nó sau self.delay milliseconds
            self.anim_job = self.root.after(self.delay, self.run_animation_step)
    except StopIteration:
        self.log("Thuật toán kết thúc.")
```

3. Tiêu chuẩn hoàn thành (Definition of Done - DoD)

Một phần việc được coi là hoàn thành 100% khi đáp ứng đầy đủ các tiêu chuẩn học thuật và lập trình sau:

- Tính nhất quán lý thuyết (Deterministic): BFS/DFS và các lựa chọn đỉnh lân cận bắt buộc phải thực hiện sắp xếp nhãn tăng dần trước khi đưa vào hàng đợi/ngăn xếp. Kết quả chạy trên máy phải trùng khớp tuyệt đối với việc làm bài tập bằng tay.
- Bất ngờ ngoại lệ và Tính toàn vẹn: Đồ thị có hướng/vô hướng được kiểm tra tính hợp lệ trước khi bấm chạy thuật toán (Ví dụ: Kruskal chỉ chạy trên vô hướng, Edmonds-Karp yêu cầu chọn rõ đỉnh nguồn và đỉnh đích).
- Clean Code & Tài liệu: Đặt tên biến rõ ràng theo tiếng Anh học thuật. Có docstring mô tả chi tiết đầu vào và đầu ra. Tất cả code gộp vào main phải được kiểm thử chéo và không gây lỗi xung đột mã nguồn.